# Project SWIFT: Product Vision & Strategy

Foundation & The Product Family

| | |
|---|---|
| **Document Title:** | **Executive Summary: Redefining SWIFT** |
| **Client:** | Karun Hitech Solutions |
| **Prepared by:** | Mohammad Sohrabipour |
| **Access Level:** | Internal / ~~External~~ / ~~Public~~ |
| **Confidentiality:** | ~~Normal~~ / ~~Confidential~~ / **Top Secret** |
| **Document ID:** | SWIFT-PV-V02-20260518 |
| **Date:** | 2026/05/18 |

# Contents

Executive Summary: Redefining SWIFT ..... 2

1 The Engineering Data Bottleneck ..... 3

1.1 The Hidden Cost: Manual Processes and Unstructured Routing ..... 3

1.2 The Flawed Alternative: Macros Tied to the UI ..... 3

1.3 The Invisible Cost: What Cannot Exist Today ..... 4

2 What SWIFT Actually Is ..... 6

2.1 Two Layers, Not One ..... 6

2.2 The Abstraction Engine ..... 7

2.3 Immunity to Updates ..... 8

3 The SWIFT Product Family ..... 10

3.1 Pillar 1 — SWIFT Automation Tools ..... 10

3.2 Pillar 2 — SWIFT Web Services (REST API) ..... 11

3.3 Pillar 3 — SWIFT Server (Data Pipelines & Analytics) ..... 12

3.4 Pillar 4 — SWIFT AI (The “SwiftMate” Integration) ..... 12

# Executive Summary: Redefining SWIFT

Before discussing roadmaps, products, or returns, we must first settle a question of identity. SWIFT is frequently described in a way that makes it sound like every other tool on the engineer's desktop. It is not. The remainder of this document depends on a single, strict definition.

**The Strict Definition.** SWIFT is, and only is, the **Universal Abstraction Layer** on top of SolidWorks and PDM. It is the engine that converts brittle, legacy CAD APIs into clean, stable data models. It is not an exporter. It is not a UI. It is not a feature pack. Everything the business will eventually "use" is built *on top of* SWIFT—not inside it.

**The Product Family Concept.** End-users do not "use" SWIFT directly, in the same way developers do not "use" a database engine—they use the applications written against it. SWIFT serves as the foundation that unlocks a family of distinct, parallel software products. Each product targets a different audience (engineers, the business, analysts, AI agents), but every product speaks to SolidWorks and PDM through the same single, stable layer.

**The Value Proposition.** By building the abstraction layer once, we can develop multiple robust products—Automation Tools, Web APIs, Data Servers, and AI integrations—without ever touching the messy underlying CAD APIs again. The cost of the abstraction is paid once. The leverage we gain from it compounds with every new product line we ship.

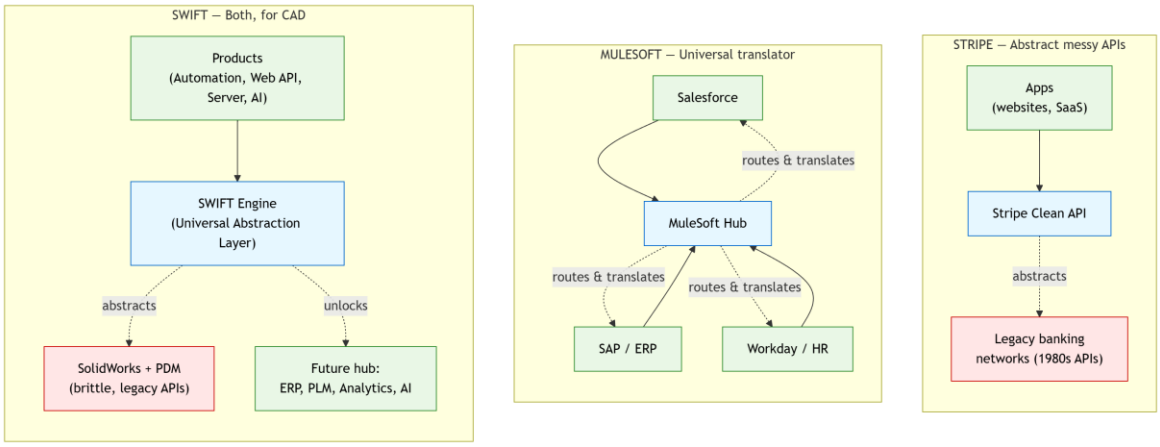


***Figure 1:*** *Stripe, MuleSoft, and SWIFT solve the same shape of problem. Stripe and SWIFT both abstract a brittle legacy API behind a clean surface; MuleSoft acts as a universal translation hub between systems that do not natively speak the same language. SWIFT is the abstraction layer today, and the foundation that unlocks the MuleSoft-style hub tomorrow.*

# 1 The Engineering Data Bottleneck

Engineering data is the most valuable, most trusted, and most reused asset in a manufacturing company. It is also the most difficult to access. Inside the CAD vault, data is rich and authoritative; outside the vault, every workflow that wants to consume that data is forced to claw it out one click at a time. This gap—between where the data lives and where it needs to go—is the single largest source of wasted engineering effort in the company.

## 1.1 The Hidden Cost: Manual Processes and Unstructured Routing

The most visible symptom of this gap is the engineer who spends multiple hours assembling a single manufacturing package. The work itself is mechanical: open an assembly, walk the BOM, locate each drawing, export a PDF, generate a STEP, build a flat pattern, name the files according to an internal convention, drop them onto the right shared folder, and notify the shop. None of these steps requires engineering judgment. All of them require expensive engineering time.

Multiply this by every release, every revision, and every project, and the hidden cost becomes structural. We are paying highly skilled mechanical engineers to act as file clerks. Worse, because the work is manual, it is also **unreliable**: a missed drawing, an outdated revision, or a mis-named STEP file silently propagates downstream until manufacturing discovers the mismatch on the shop floor—typically by scrapping a part.

## 1.2 The Flawed Alternative: Macros Tied to the UI

The industry's standard response to this bottleneck is to write a macro. Macros and scripts feel like the right solution: they automate the clicks, they live next to the CAD software, and they can be authored by a clever internal user. In practice, macros are the worst of both worlds.

A macro is tied directly to the SolidWorks UI rather than to an abstraction of the underlying data. It is, in effect, a recording of a particular engineer's mouse and keyboard at a particular point in time. Three things follow inevitably from this:

- **They break with every update.** Each SolidWorks or PDM release changes the UI, the COM surface, or both. Every change risks invalidating the macro in ways the original author may not even notice.
- **They fail silently.** When a macro hits an unexpected state, it rarely raises a clean error—it skips the offending item, finishes with what looks like a success, and lets bad data flow downstream.

- **They do not scale across products.** A macro built for one report cannot be reused by an ERP integration, a web dashboard, or an AI agent. The work is local, brittle, and trapped on a single machine.

The lesson is that the bottleneck cannot be solved *above* the CAD API, where macros live. It must be solved *at* the CAD API, by replacing the unstable surface with a stable one. That is what SWIFT does.

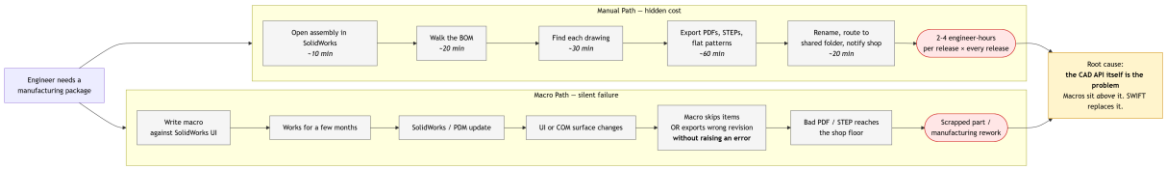


***Figure 2:*** *The two ways the current bottleneck plays out. The manual path burns engineer-hours every release. The macro path appears to fix this, but inherits the instability of the underlying CAD API and silently propagates bad data to the shop floor. Both paths share a single root cause: the CAD API itself is what needs to be replaced.*

## 1.3 The Invisible Cost: What Cannot Exist Today

The manual workflows and brittle macros described above are the *visible* costs of the bottleneck. They are easy to point at, easy to measure, and easy to argue for. But they are not the whole story. Beneath them sits a much larger invisible cost: an entire category of capabilities that simply does not exist for engineering data today—not because the tooling is missing, but because the data cannot be reached.

Consider what the rest of the business takes for granted. Finance plugs a BI tool into a clean accounting database and gets dashboards on day one. Sales connects an ERP to a reporting layer and gets pipeline analytics without writing code. Operations integrates HR, payroll, and time-tracking through standard connectors. None of these teams build their own dashboards, their own pipelines, or their own AI assistants from scratch. They buy them off the shelf, point them at a clean data layer, and start working.

Engineering has none of this. Not because the tools do not exist—Power BI, Tableau, Metabase, Airflow, modern LLM agents would all happily consume engineering data—but because there is no clean data layer to point them at. So the capabilities simply do not exist. The gap is not solved by working harder inside CAD; it is solved by giving every other tool a way *into* CAD.

The industry's standard answer to this gap is to sell a monolithic platform that does everything itself: SAP, 3DEXPERIENCE, Teamcenter, and similar. These platforms cost hundreds of thousands of dollars per year, take years to deploy, and lock the company into a single vendor—precisely because they include their own private data layer that nothing else can reach. They are expensive because they are the only way in.

SWIFT inverts this trade-off. By being the abstraction layer itself, SWIFT makes engineering data accessible to the off-the-shelf tools the rest of the business already trusts. We do not build dashboards—we point an existing BI tool at the SWIFT Server. We do not write ERP integrations from scratch—we expose the SWIFT API and let standard connectors do the work. We do not develop an AI copilot—we let an existing LLM agent call SWIFT's MCP tools.

The result is that SWIFT closes both gaps at once. It eliminates the visible costs of manual work and brittle macros, and it unlocks the much larger set of capabilities that previously could not exist at any price short of a six-figure platform purchase.

# 2 What SWIFT Actually Is

Most attempts to “modernize CAD” fail because they try to build a better exporter on top of the same broken API. SWIFT inverts the problem. We do not build a better tool. We build the layer that makes building any number of tools cheap, safe, and reliable.

## 2.1 Two Layers, Not One

The single most important architectural idea behind SWIFT is the separation between the **engine** and the **products**.

- The **engine**—SWIFT itself—talks to SolidWorks and PDM. It absorbs every quirk, retry, COM error, and undocumented behavior of those APIs. From the outside, the engine exposes a clean, predictable surface: typed data, explicit results, no surprises.
- The **products**—Automation Tools, Web APIs, Data Servers, AI integrations—talk only to the engine. They never see SolidWorks or PDM. They never know which version is installed, which add-in is active, or which COM call is currently misbehaving.

This separation is not stylistic; it is the entire reason the product family exists. Without it, every product would carry its own copy of CAD-handling code, and every product would break on its own schedule.

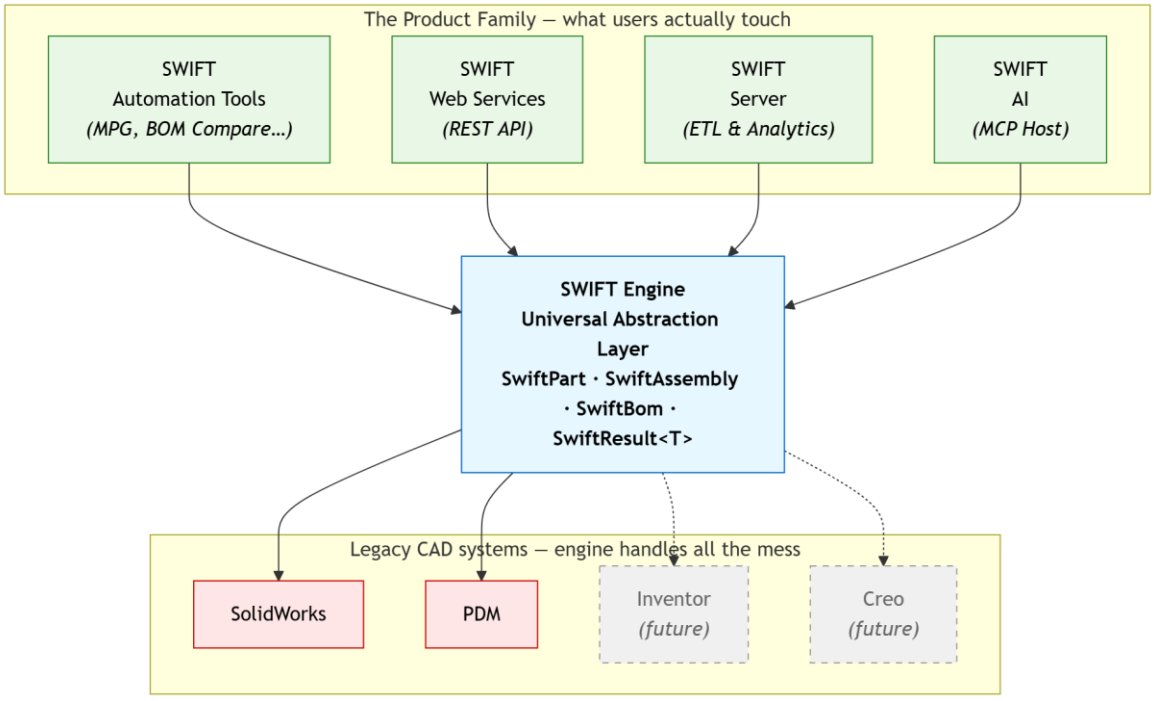


***Figure 3:*** *Two layers, not one. The product family sits on top; the SWIFT engine sits in the middle and owns every interaction with the underlying CAD systems. SolidWorks and PDM are in scope today; Inventor and Creo are shown dashed because the same architecture can absorb them later through additional adapters, with no change required in any of the products above.*

## 2.2 The Abstraction Engine

Concretely, SWIFT maps raw PDM and SolidWorks data into clean, well-typed objects. Where the underlying CAD API offers a tangle of partially-initialised COM handles, SWIFT offers a small, focused vocabulary:

- `SwiftPart` — a single CAD part, with the metadata, custom properties, configurations, and file identity that the business actually cares about.
- `SwiftAssembly` — a structured product assembly, with its components, usage links, and the relationships needed to walk or analyse the tree.
- `SwiftBom` — a clean, analysed Bill of Materials, with revisions, quantities, drawings, and a deterministic structure that any downstream consumer can rely on.
- `SwiftResult<T>` — the result type returned by every operation. It either succeeds with data, or it explicitly fails with a typed error. There is no third state. There are no silent failures.

These objects live in `SWIFT.Core.Abstractions` and `SWIFT.Core.Domain`. Everything else in the SWIFT codebase—PDM adapters, SolidWorks adapters, the REST host, the MCP host, the WinForms shell—exists to either produce these objects or to consume them. They are the shared vocabulary that every SWIFT-based product is built against.

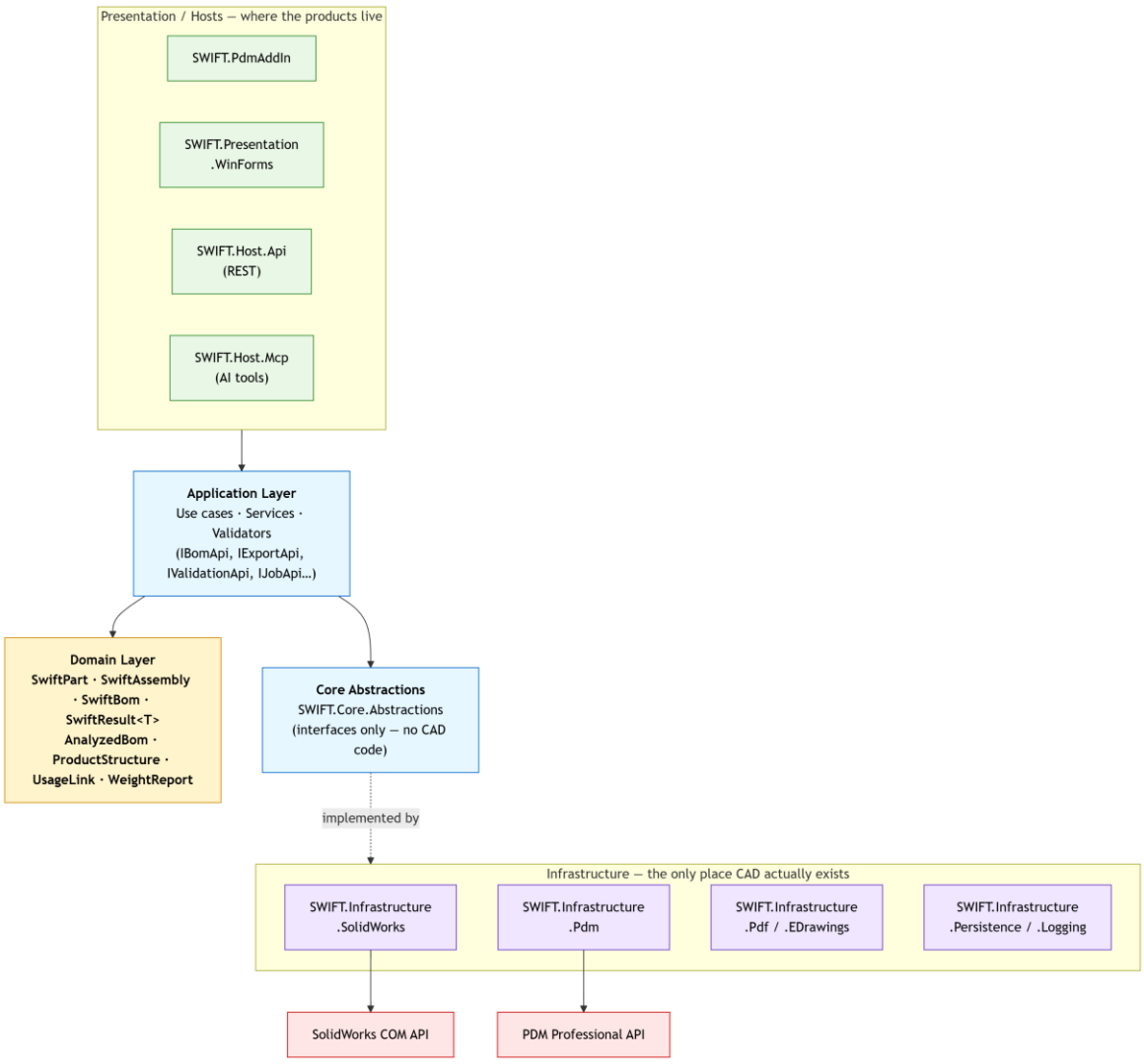


***Figure 4:*** *Inside the engine: a Clean Architecture in which the domain vocabulary (`SwiftPart`, `SwiftAssembly`, `SwiftBom`, `SwiftResult<T>`) is the single source of truth. Hosts at the top (PDM Add-in, WinForms, REST API, MCP server) all consume the same application services, and the only place that touches the raw SolidWorks and PDM APIs is the Infrastructure layer at the bottom. Replacing a CAD vendor means writing one new adapter—never touching the domain or any product above it.*

## 2.3 Immunity to Updates

Because SWIFT abstracts the API, future CAD updates do not break the downstream products. When SolidWorks ships a new version or PDM changes its behavior, only one place in the system—the relevant adapter inside the engine—needs to change. The products built on top of SWIFT continue to ask for the same `SwiftBom`, the same `SwiftAssembly`, the same `SwiftResult<T>`, and they continue to get the same answers.

This is the same playbook that Stripe used to abstract the legacy banking network: *don't talk to the banks; talk to us*. SWIFT applies it to manufacturing data: don't talk to SolidWorks; talk to

SWIFT. The messy translation happens in one place, maintained by one team, and every product built on top of it inherits the stability automatically.

# 3 The SWIFT Product Family

With the foundation defined, the product strategy becomes a straightforward question: *what is now possible that was not possible before?* The answer is a family of four distinct product lines, each targeting a different audience and each independently shippable.

These pillars are not phases of a single product, and they are not steps in a sequence that must be completed in order. Because they all stand on the same abstraction layer, they can be developed in parallel, each with its own roadmap, its own release cadence, and its own commercial life.

## 3.1 Pillar 1 — SWIFT Automation Tools

**Product Focus.** Desktop-level engineering efficiency. These are the tools an engineer launches from inside their normal workflow—inside PDM, inside SolidWorks, or from a standalone shell—to eliminate manual, repetitive work.

**The Flagship Tool (v0.1).** The **Manufacturing Package Generator (MPG)**. The MPG takes an approved assembly and produces a complete, correctly named, correctly routed manufacturing package: PDFs, STEPs, BOMs, flat patterns, and any other artifacts the company's release standard requires.

**Key Capabilities.**

- Standardised export of PDFs, STEPs, and BOMs against a configurable company profile.
- **Zero silent failures.** Every operation flows through `SwiftResult<T>`: the package is either complete and correct, or the run halts and reports exactly which item failed and why.
- Deep integration with PDM workflows via the `SWIFT.PdmAddIn` surface, and a desktop UI via `SWIFT.Presentation.WinForms`.

**Future Scope.** BOM comparison tools, quality-control enhancements (pre-release validation, naming and revision checks), and broader release automation across the engineering workflow.


***Figure 5:*** *Pillar 1 in operation. An engineer triggers the Manufacturing Package Generator from inside the PDM context menu; the SWIFT engine produces the full set of release artifacts—PDFs, STEPs, BOM, flat patterns, routed and named to standard—or halts with a typed error. There is no partial output: the package is either complete and correct, or it is explicitly rejected before anything reaches the shop floor.*

## 3.2 Pillar 2 — SWIFT Web Services (REST API)

**Product Focus.** Unlocking engineering data for the rest of the business. Today, anyone outside engineering who wants CAD or PDM data has two options: ask an engineer to export it, or install SolidWorks themselves. Pillar 2 removes both.

**The Flagship Tool.** The **SWIFT REST API Foundation**, hosted by `SWIFT.Host.Api`. The API exposes the same `SwiftBom`, `SwiftAssembly`, and `SwiftPart` objects over HTTP, behind proper authentication, authorization, and audit controls.

**Key Capabilities.**

- Authorised web applications and enterprise systems can request engineering data securely, without requiring a SolidWorks license or a local installation.
- Focused, well-scoped endpoints—BOM, Export, Files, Configuration, Validation, Manufacturing Package, Jobs, Health, and Capabilities—each backed by a corresponding controller.
- Long-running operations are modelled explicitly as jobs, with tracking and audit, rather than as fire-and-forget calls.

**Future Scope.** Direct ERP endpoints (NetSuite, SAP, and similar), BOM exchange workflows for purchasing and production, and supplier-facing portals that expose a curated slice of engineering data to external partners.

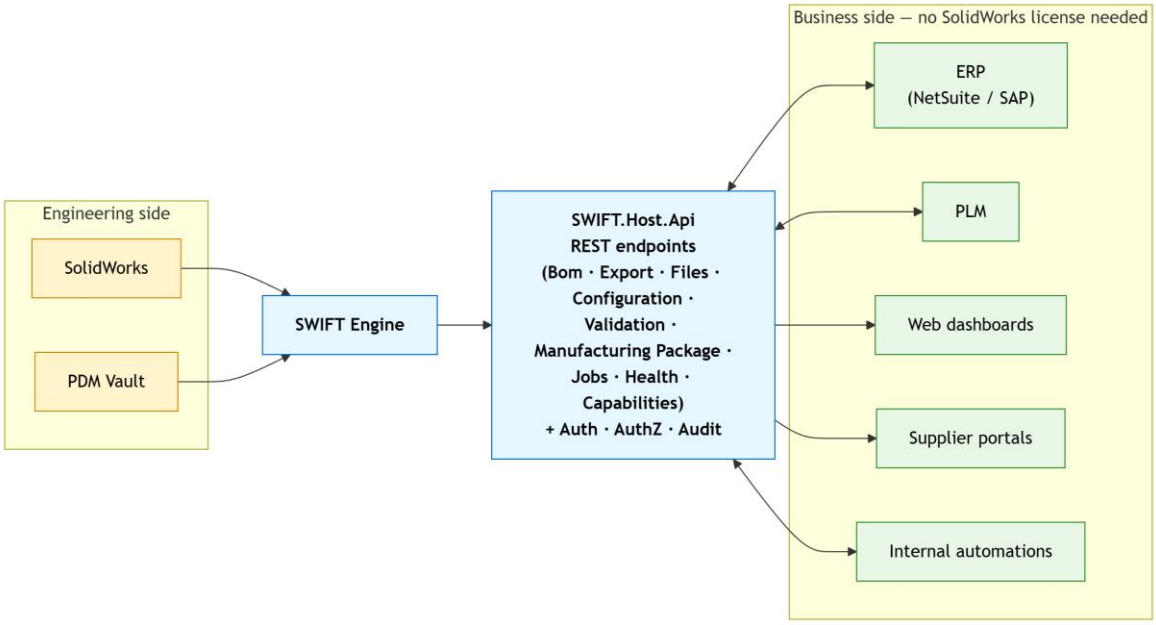


***Figure 6:*** *Pillar 2 in operation. The SWIFT REST host sits between the engineering side (SolidWorks and PDM, accessed only through the engine) and the business side (ERP, PLM, web dashboards, supplier portals, internal automations). Authentication, authorization, and audit live at the API boundary; consumers on the business side never need a SolidWorks license to read or act on engineering data.*

## 3.3 Pillar 3 — SWIFT Server (Data Pipelines & Analytics)

**Product Focus.** Engineering business intelligence. PDM was designed to store files, not to answer analytical questions. Querying it heavily slows the system down for the engineers who depend on it. Pillar 3 takes the data out of the vault—safely, on a schedule—and lands it somewhere the business can actually analyse.

**The Flagship Tool. Nightly ETL Pipelines and the SWIFT Server**, built on the job runtime inside `SWIFT.Host.Api`.

**Key Capabilities.**

- **State-aware extraction.** Instead of querying the live PDM vault item-by-item and slowing down engineers, the SWIFT Server looks at PDM *states*. If a file's state has not changed since the last run, no ETL occurs for that file. If it has, the engine extracts the clean abstraction data and pushes it to a separate analytical database.
- Clean data, by construction. Because the pipeline emits the same `SwiftBom` / `SwiftAssembly` / `SwiftPart` objects, the downstream warehouse never has to deal with raw CAD formats.
- Zero impact on live engineering performance: the heavy reads happen once, on a schedule, in the background, and only against material that has actually changed.

**Future Scope.** Web dashboards for management—release velocity, revision activity, BOM churn, weight and cost trends—powered by the clean semantic data, with a feel similar to an internal analytics server but backed by data we trust.

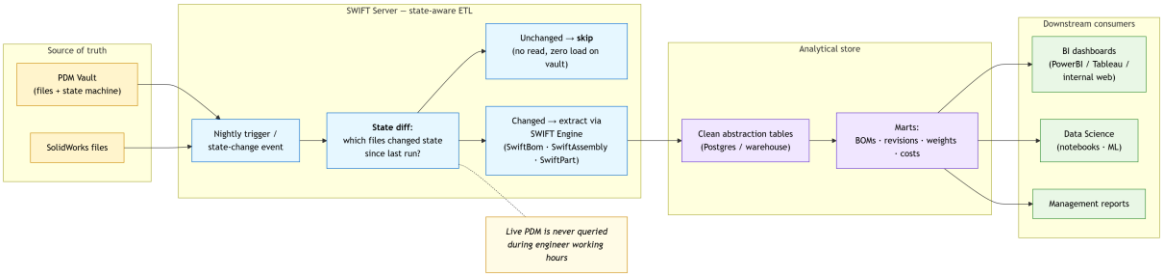


***Figure 7:*** *Pillar 3 inside a modern data stack. The SWIFT Server starts from the PDM state machine, computes a diff of what has actually changed since the last run, and re-extracts only those files through the SWIFT engine. Clean abstraction data lands in a separate analytical store feeding BI dashboards, data-science notebooks, and management reports. The live vault is never queried during engineer working hours.*

## 3.4 Pillar 4 — SWIFT AI (The "SwiftMate" Integration)

**Product Focus.** Autonomous workflow agents. Large Language Models have become extremely capable at reasoning over structured data and at calling well-typed tools. They are, however,

fundamentally incapable of clicking through a SolidWorks menu reliably. They need an interface that speaks JSON, not pixels.

**The Flagship Tool.** The **SWIFT MCP (Model Context Protocol) Host**, implemented by `SWIFT.Host.Mcp`. The MCP host exposes SWIFT's operations—BOM extraction, export, validation, configuration, file operations, job control—as a catalogue of typed tools that any MCP-compatible AI agent can call.

**Key Capabilities.**

- LLMs cannot click SolidWorks menus, but they can read SWIFT's clean JSON and trigger SWIFT's safe API endpoints.
- Every tool call is grounded in the same abstraction objects the REST API and the desktop tools use; an agent's view of a BOM is the same view a human user gets.
- Determinism by construction: tools either succeed, or they return a typed failure. An agent never has to guess whether an operation "mostly worked".

**The SwiftMate Relationship.** This is where the SWIFT foundation meets the separate **SwiftMate** project. SwiftMate is an LLM-driven SolidWorks copilot—an interactive assistant that lets an engineer ask questions and request actions in natural language. SwiftMate lives in its own repository, with its own architecture, and its own roadmap. It is *not* part of SWIFT.

The two systems meet through Pillar 4. SwiftMate consumes SWIFT's MCP host as a grounding layer: when the agent needs to reason about a BOM, inspect a configuration, or trigger an export, it does so through SWIFT's typed, mathematically-guaranteed tools rather than by trying to drive SolidWorks directly. SWIFT keeps SwiftMate honest; SwiftMate gives SWIFT a voice.

**Future Scope.** A growing operation catalogue that lets SwiftMate (and other agents) answer increasingly sophisticated questions about engineering data and safely trigger an expanding set of automation workflows—always through the same abstraction layer, never against the raw CAD APIs.

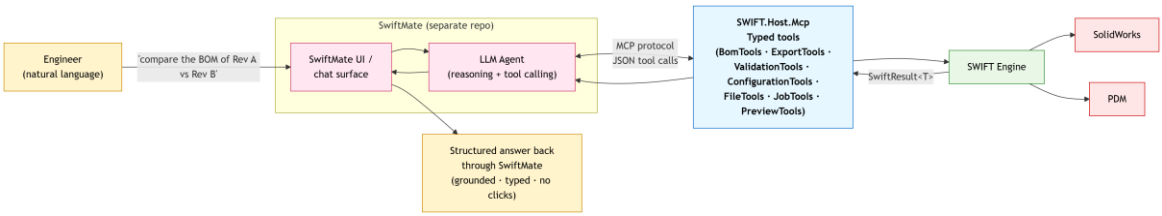


***Figure 8:*** *Pillar 4 end-to-end. An engineer asks SwiftMate—a separate product, in its own repository—a question in natural language. SwiftMate's LLM agent translates the request into typed MCP tool calls against `SWIFT.Host.Mcp`, which executes them through the SWIFT engine. Every step is grounded in the same abstraction objects the desktop and REST products use; the agent never has to drive SolidWorks through its UI.*